AWS — Production-Ready VPC + Private RDS (MySQL)

Complete, step-by-step implementation guide (console + CLI + verification + hardening + testing + cleanup). Replace placeholder values (<...>) with your own names/IDs.

Quick project summary / goals

- Create an isolated, production-style VPC with public and private subnets (multi-AZ ready).
- Deploy RDS (MySQL) in private subnets (no public IP).
- Deploy an EC2 bastion/app server in the public subnet to access RDS.
- Implement route tables, security groups, IAM role for secrets, backups, encryption, and cleanup.
- Deliverables: working RDS accessible only from bastion, automated backups, monitoring & alerts, secure credential handling via Secrets Manager.

Table of contents

    - RDS instance (MySQL) — production options (Multi-AZ, encryption, monitoring)
    - IAM role & Secrets Manager (EC2 reads DB credentials)
    - EC2 bastion/app server (attach role & test)
    - Optional: Snapshot export to S3 (IAM role + export task)
5. Tests & validation checklist (exact commands/SQL)
6. Troubleshooting common errors & fixes
7. Cleanup steps (console + CLI)

1) Naming & placeholders (replace everywhere)

Use a consistent prefix: `proj` or `harshitha-rds`
Examples used below:

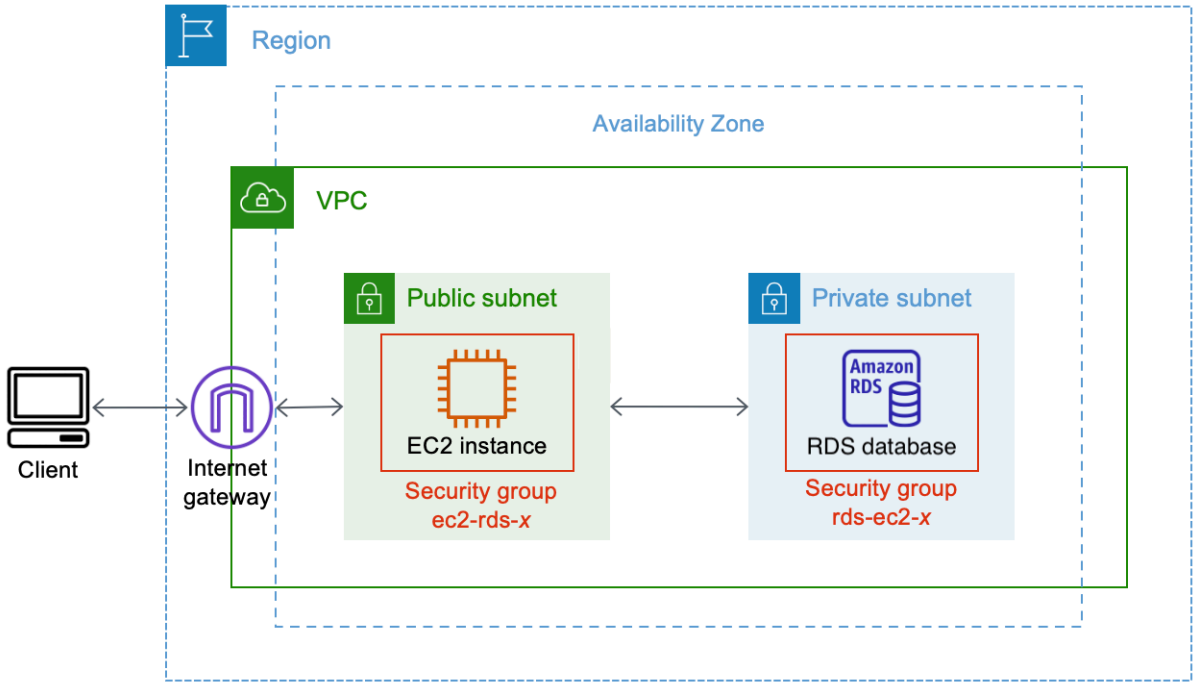
- VPC: `prod-vpc`
- Public subnet(s): `prod-public-a` (CIDR `10.0.1.0/24`), `prod-public-b` (optional)
- Private subnets: `prod-private-a` (`10.0.1.0/24`), `prod-private-b` (`10.0.2.0/24`)
- Internet Gateway: `prod-igw`
- Route tables: `prod-public-rt`, `prod-private-rt`
- Security groups: `sg-bastion`, `sg-rds`
- DB subnet group: `prod-db-subnet-group`
- RDS instance id: `prod-rds-db`
- EC2 bastion: `prod-bastion`
- Secrets Manager secret name: `prod/rds/harshi`
- IAM role for EC2: `EC2SecretsRole`

- S3 exported snapshot bucket: `prod-rds-exports-<suffix>`
- Region: `<REGION>` (e.g., `ap-south-1`)
- Account id: `<ACCOUNT_ID>`

2) Architecture

Why this layout:

- Public subnet = bastion (only place with public IP).
- RDS in private subnets - no public endpoint.
- Internet Gateway provides internet access.
- Security groups reference each other (SG→SG) — recommended.



3) Pre-flight checks & cost control

- Use a sandbox AWS account.
- Create a budget: Billing → Budgets → create a cost budget with a low threshold ($1–$10) and email alert.
- Use small instance types for practice (`t3.micro`, `db.t3.micro`).

4) Full build — console steps (detailed) + CLI snippets

> Work in this order. For each Console section I give click-by-click; CLI snippets follow where useful.

4.1 Create VPC

Console steps

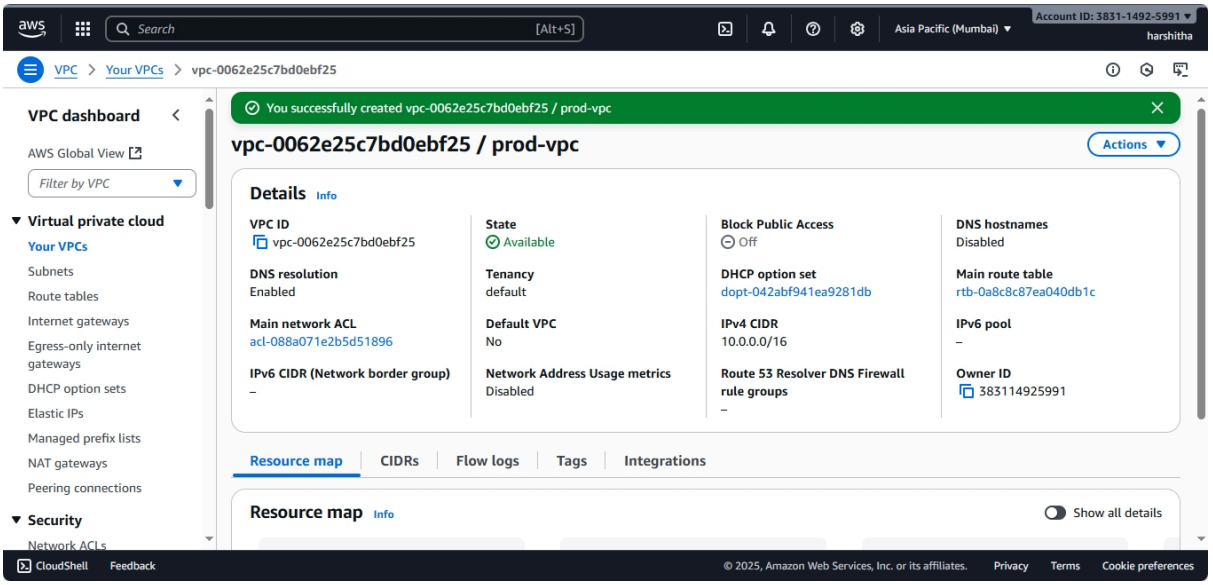
1. Services → VPC → Your VPCs → Create VPC.
2. Choose “VPC only” or “VPC and more”.
3. Name: `prod-vpc`. IPv4 CIDR: `10.0.0.0/16`. Tenancy: default.

   Create.

CLI

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=prod-vpc}]' \
  --region <REGION>
```

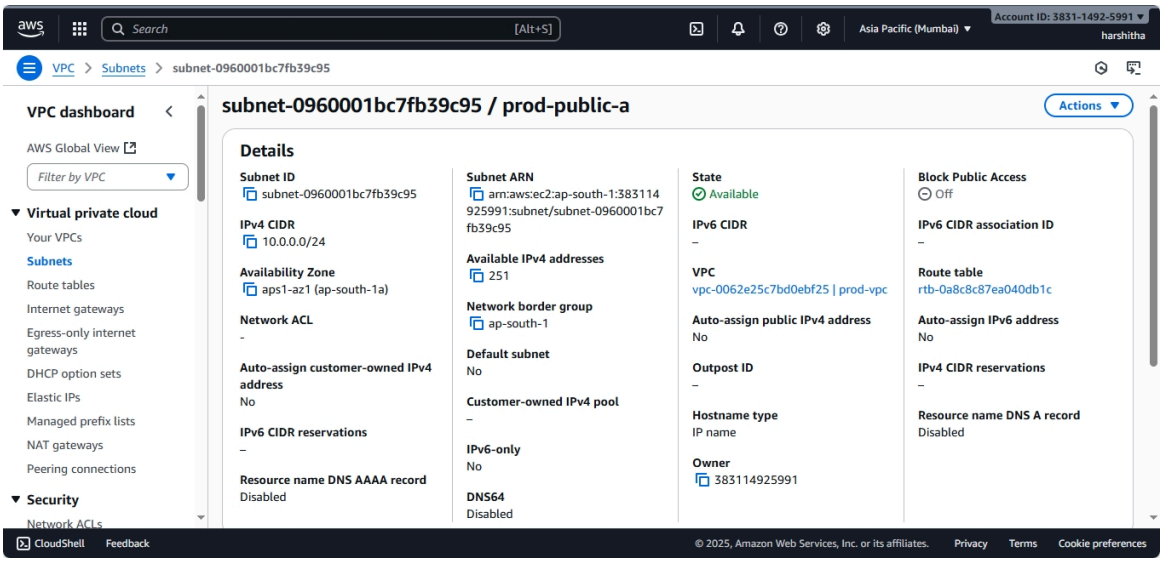
Record the returned `VpcId`.



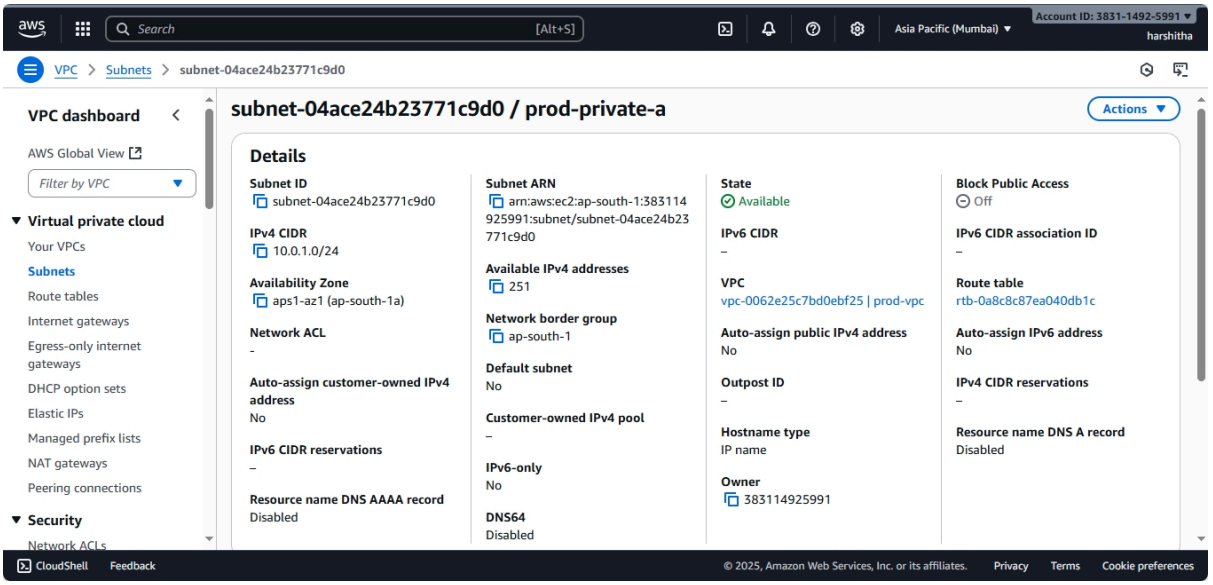
4.2 Create subnets (public + private in two AZs for HA)

Console

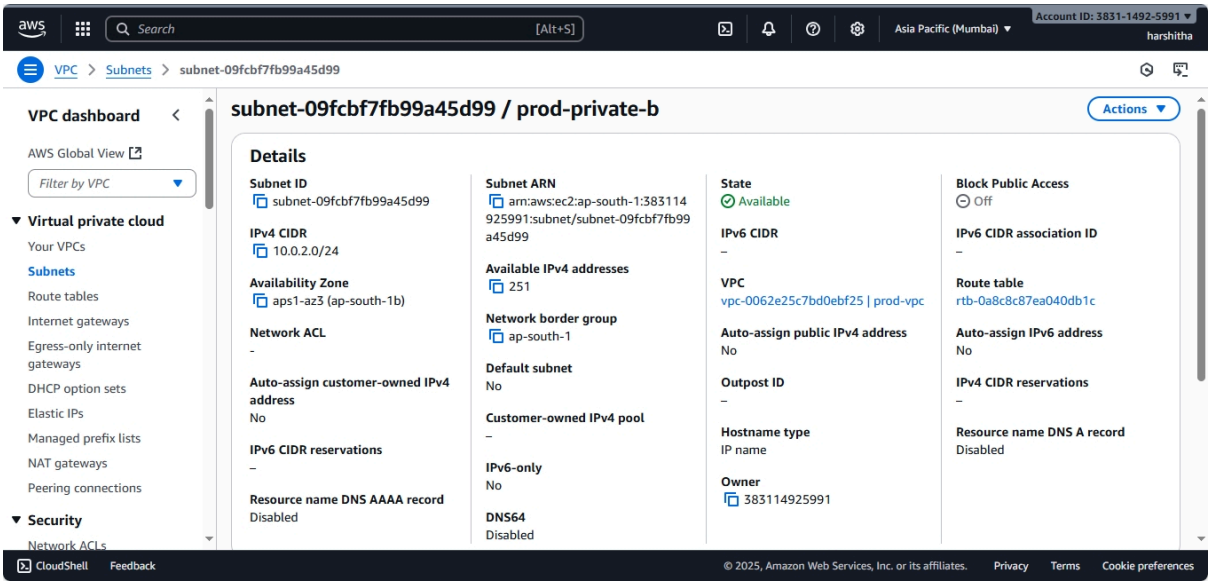
1. VPC → Subnets → Create subnet.
    - Select VPC: `prod-vpc`.
    - Name: `prod-public-a`. AZ: choose `<REGION>a`. IPv4 CIDR: `10.0.1.0/24`. Enable auto-assign public IPv4.
    - Create.



2. Create private subnet `prod-private-a` in same AZ: CIDR `10.0.1.0/24` (auto-assign disabled).



3. Repeat in another AZ for HA: `prod-private-b` (10.0.3.0/24) and `prod-public-b` (optional).



CLI (example)

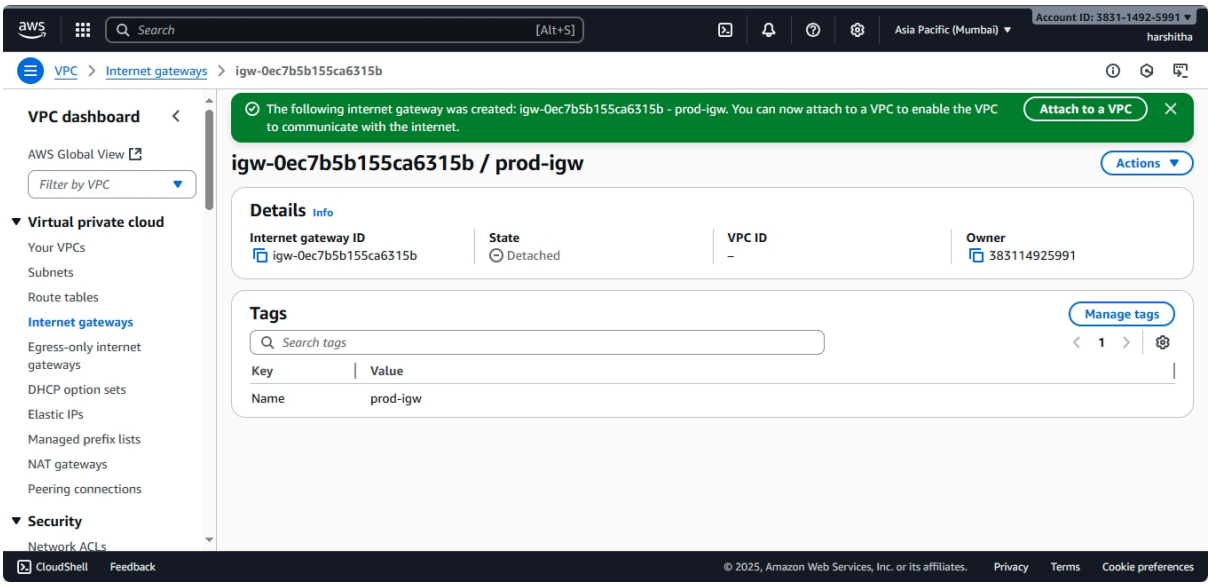
```
aws ec2 create-subnet --vpc-id <VPC_ID> --cidr-block 10.0.1.0/24 \
  --availability-zone <REGION>a --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=prod-public-a}]'
```

Repeat for each subnet.

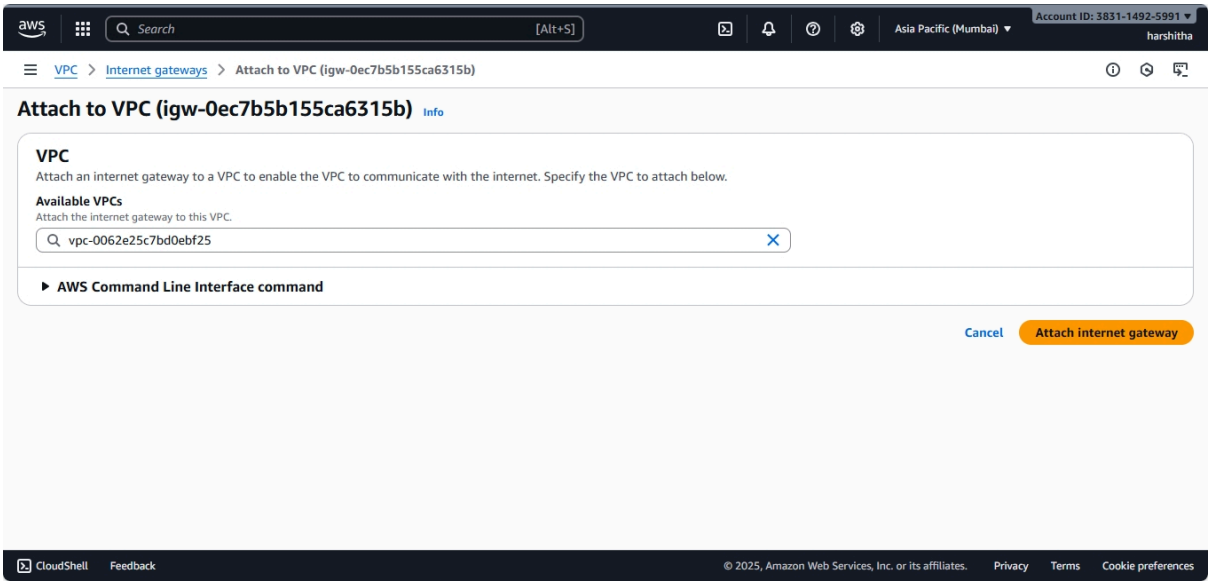
4.3 Create Internet Gateway and attach

Console

1. VPC → Internet Gateways → Create internet gateway → Name: `prod-igw`.

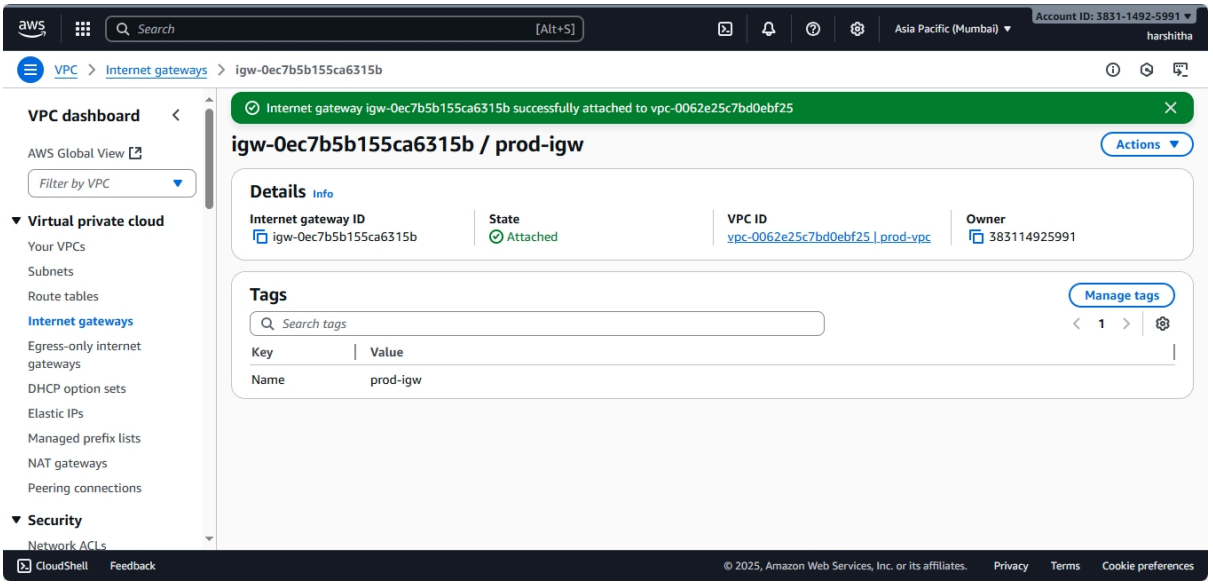


2. Select `prod-igw` → Actions → Attach to VPC → choose `prod-vpc`.



CLI

aws ec2 create-internet-gateway --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=prod-igw}]'
aws ec2 attach-internet-gateway --internet-gateway-id <IGW_ID> --vpc-id <VPC_ID>



4.4 Create NAT Gateway (public subnet) — provides outbound for private subnets(optional:if you want to allow outbound traffic to private subnet)

Console

1. VPC → Elastic IPs → Allocate Elastic IP → tag name `prod-eip`.

2. VPC → NAT Gateways → Create NAT gateway. Choose subnet `prod-public-a`, Elastic IP = `prod-eip`. Create and wait for `Available`.

Notes: NAT Gateway costs money—remove when done.

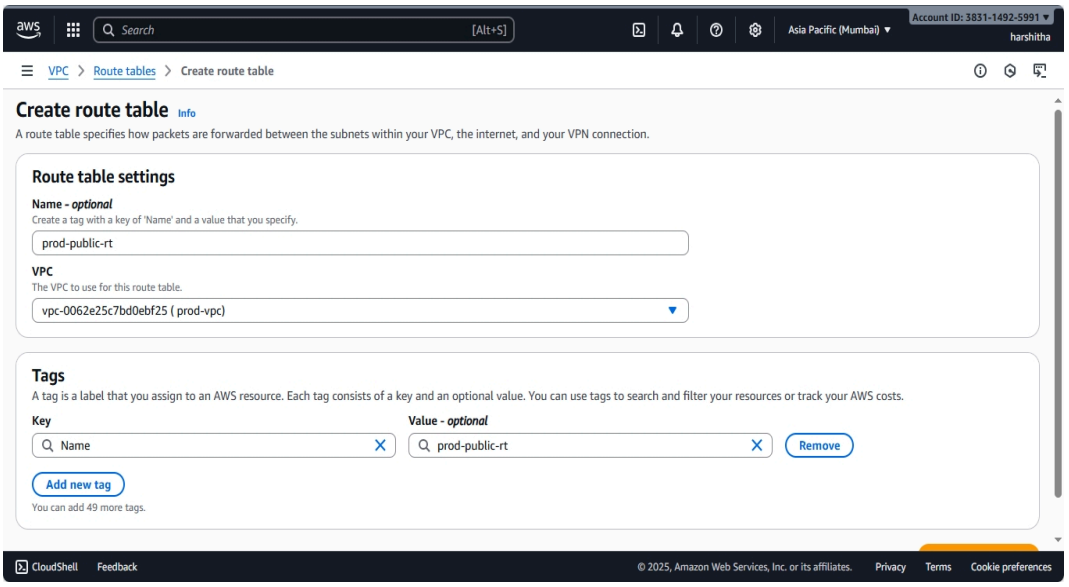
CLI (allocate EIP + create NAT)

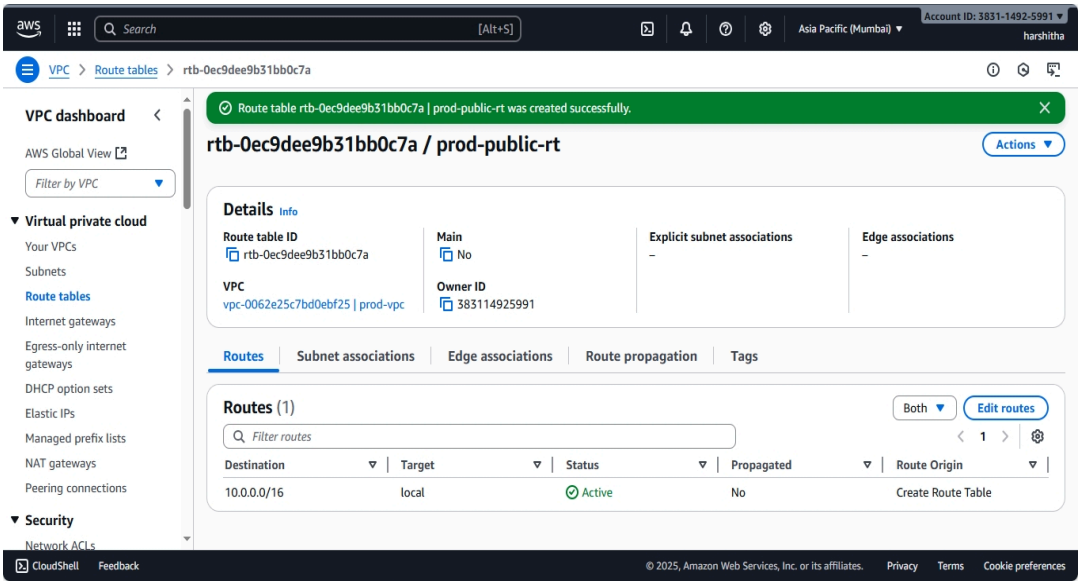
aws ec2 allocate-address --domain vpc
aws ec2 create-nat-gateway --subnet-id <PUBLIC_SUBNET_ID> --allocation-id <EIP_ALLOC_ID>

4.5 Route tables: public and private routes

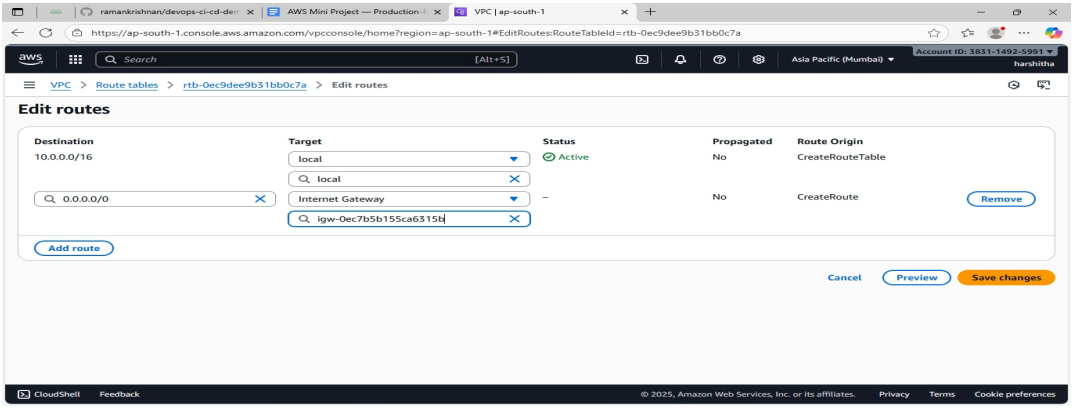
Console

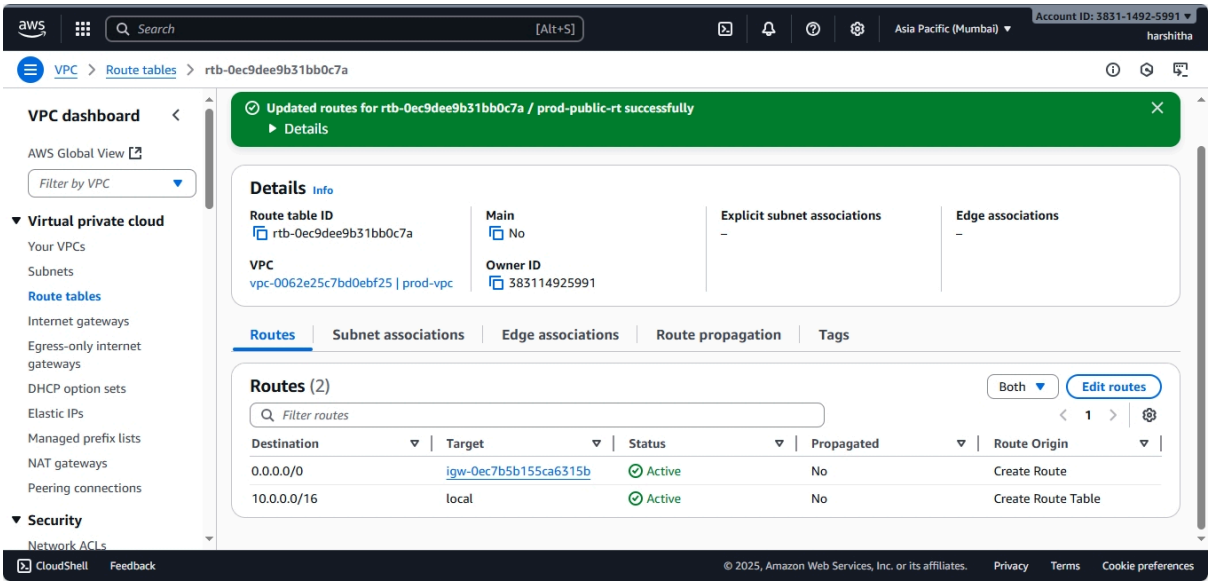
1. VPC → Route tables → Create route table → Name `prod-public-rt` → VPC `prod-vpc`.



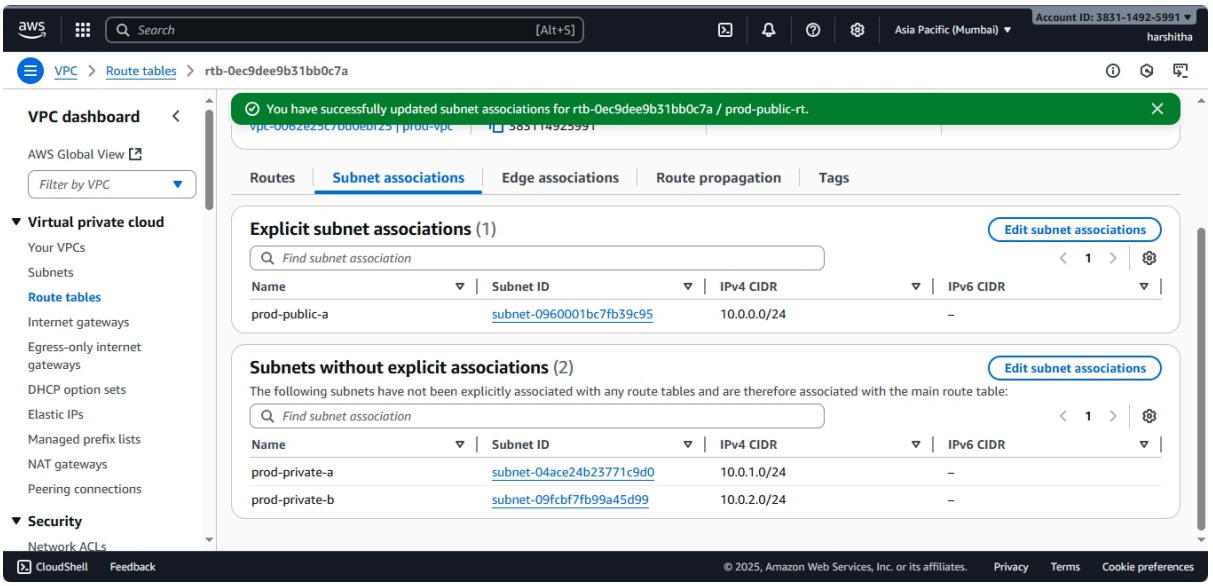


2. Select `prod-public-rt` → Routes → Edit routes → Add route `0.0.0.0/0` → Target: `Internet gateway` → choose `prod-igw`. Save.





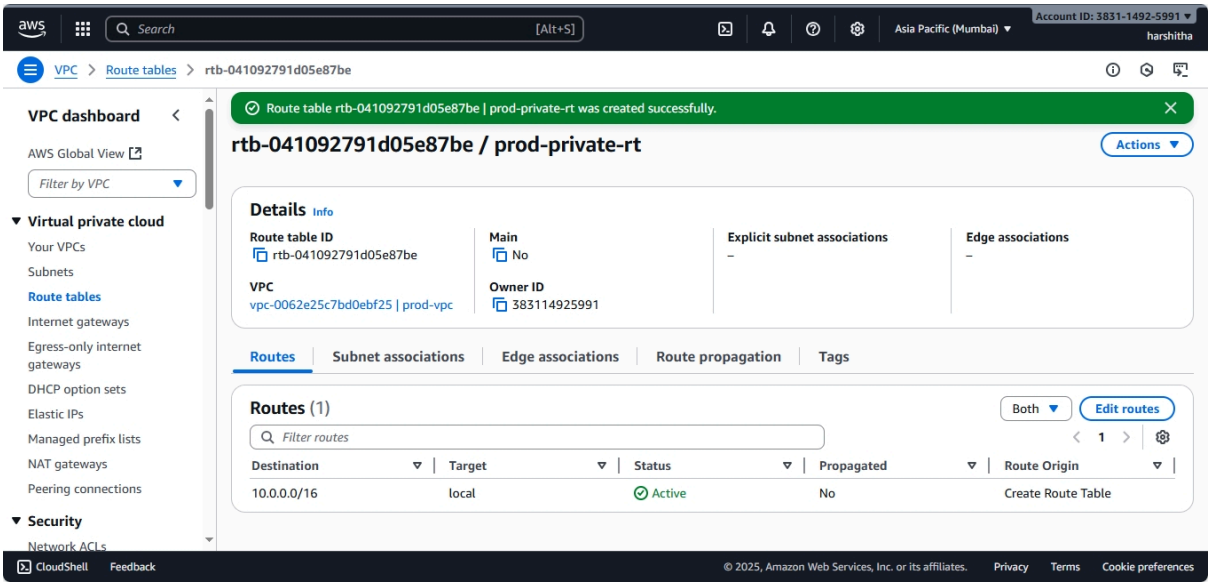
3. Select `prod-public-rt` → Subnet associations → Edit subnet associations → associate `prod-public-a` (and `prod-public-b` if exists).

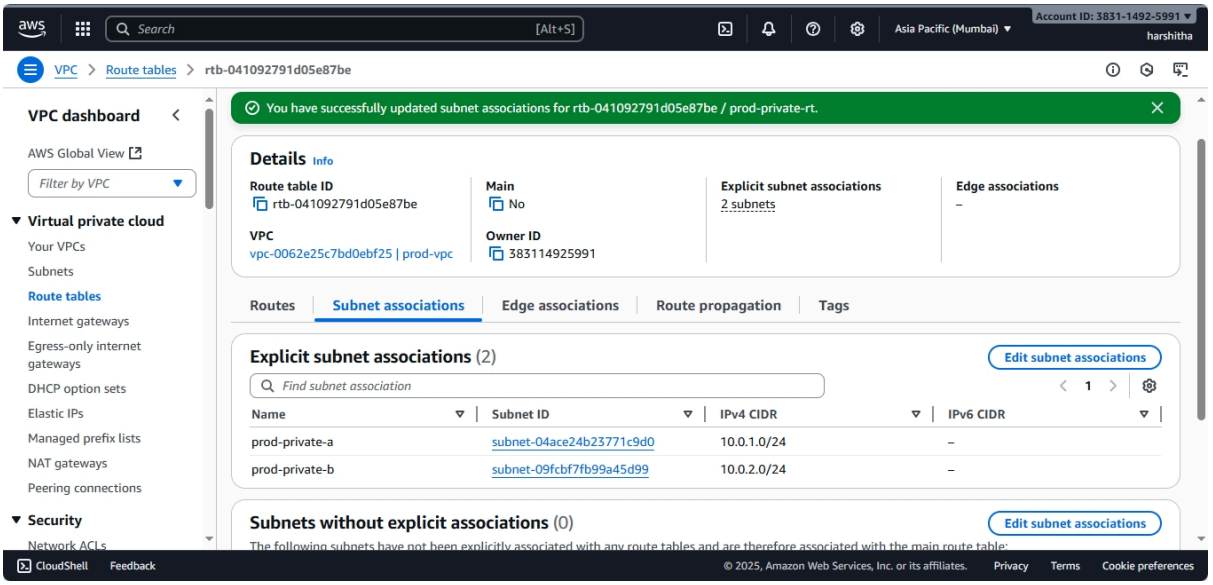


CLI

aws ec2 create-route-table --vpc-id <VPC_ID> --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=prod-public-rt}]'
aws ec2 create-route --route-table-id <PUB_RT_ID> --destination-cidr-block 0.0.0.0/0 --gateway-id <IGW_ID>
aws ec2 associate-route-table --route-table-id <PUB_RT_ID> --subnet-id <PUBLIC_SUBNET_ID>
private RT -> route to NAT
aws ec2 create-route --route-table-id <PRIV_RT_ID> --destination-cidr-block 0.0.0.0/0 --nat-gateway-id <NAT_ID>

4. Create route table `prod-private-rt` → add route `0.0.0.0/0` → Target: NAT Gateway `prod-nat`. Associate with `prod-private-a`, `prod-private-b`.



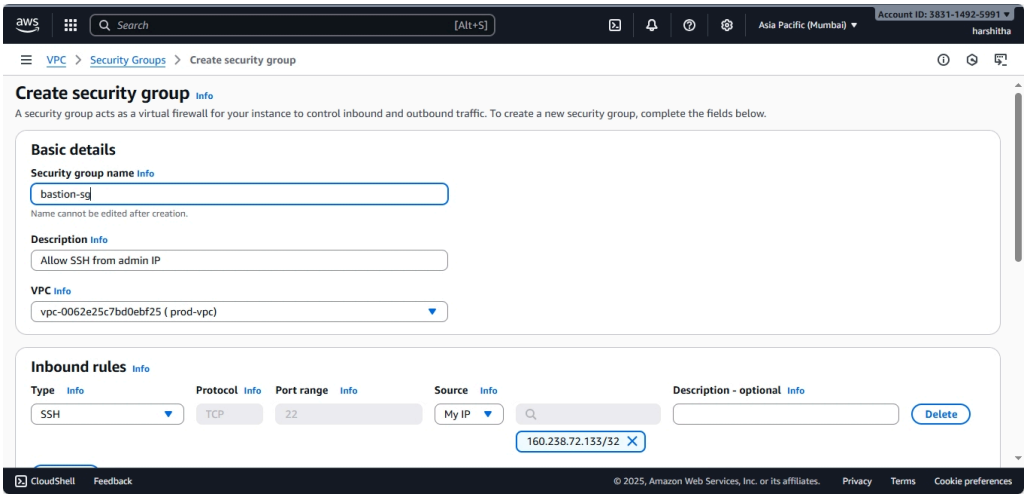


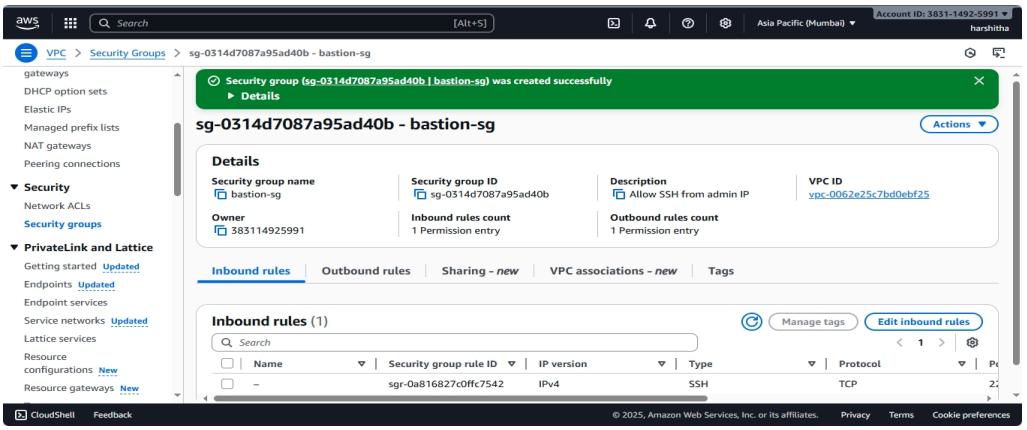
4.6 Create Security Groups

Security groups (SG) must follow least privilege and reference SGs where possible.

Console: create sg-bastion

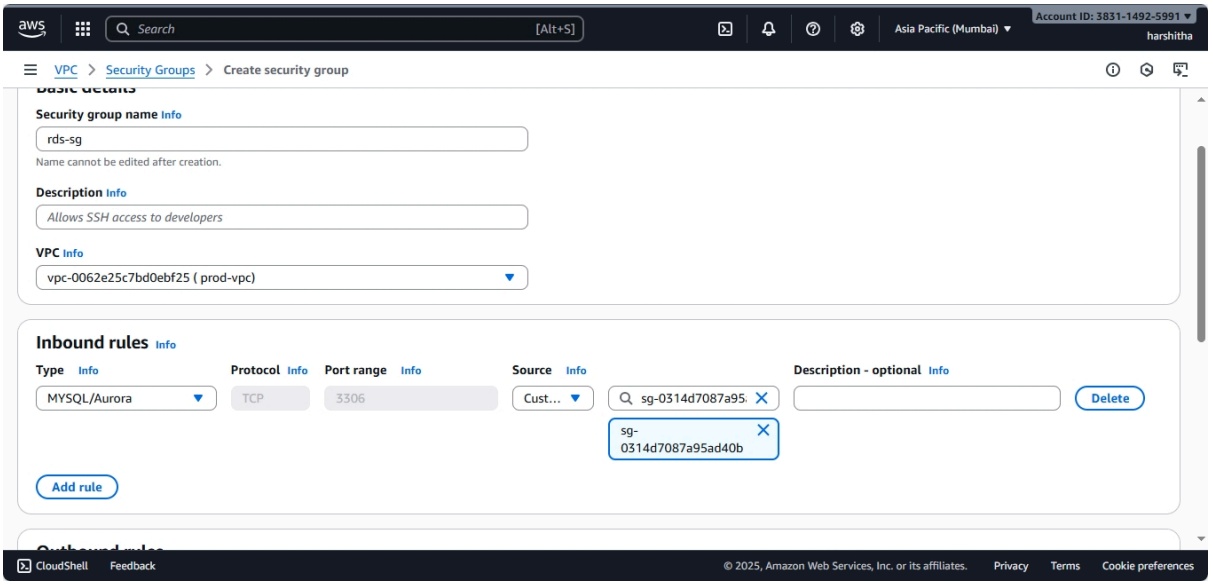
1. VPC → Security Groups → Create security group
    - Name: `bastion-sg`
    - Description: `Allow SSH from admin IP`
    - VPC: `prod-vpc`
    - Inbound rules:
        - Type: SSH (TCP 22) → Source: `My IP` (enter your public IP)
    - Outbound: Allow all (default)

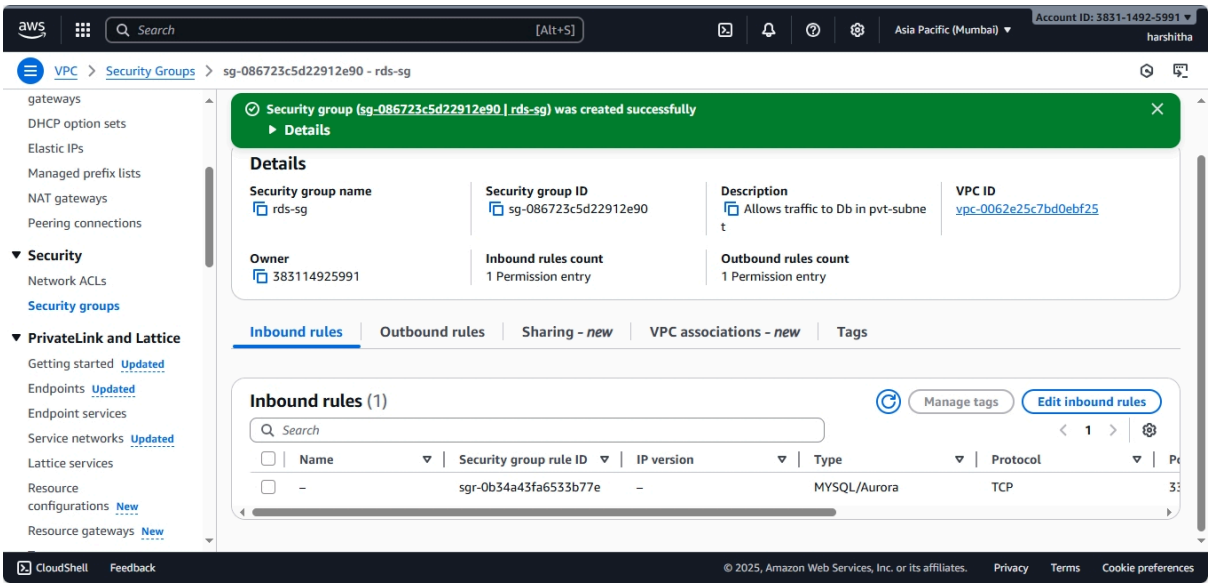




Create rds-sg

1. Create security group `rds-sg` in `prod-vpc`.
    - Inbound rules:
        - Type: MySQL/Aurora (3306) → Source: Custom → choose **security group** `sg-bastion` (this allows only instances in sg-bastion to reach the DB).
    - Outbound: Allow all.





Best practice: use SG→SG references, not IP ranges for internal traffic.

CLI (example)

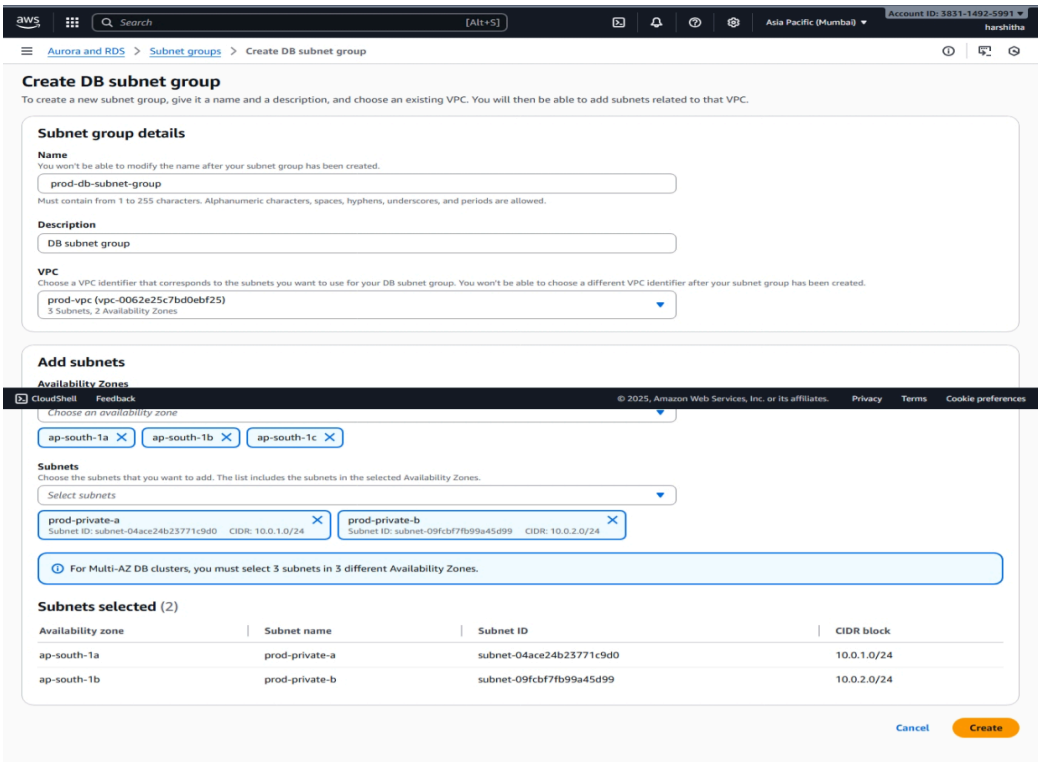
aws ec2 create-security-group --group-name sg-bastion --description "bastion SG" --vpc-id <VPC_ID>
aws ec2 authorize-security-group-ingress --group-id <SG_BASTION_ID> --protocol tcp --port 22 --cidr <YOUR_IP/32>
aws ec2 create-security-group --group-name sg-rds --description "rds SG" --vpc-id <VPC_ID>
aws ec2 authorize-security-group-ingress --group-id <SG_RDS_ID> --protocol tcp --port 3306 --source-group <SG_BASTION_ID>

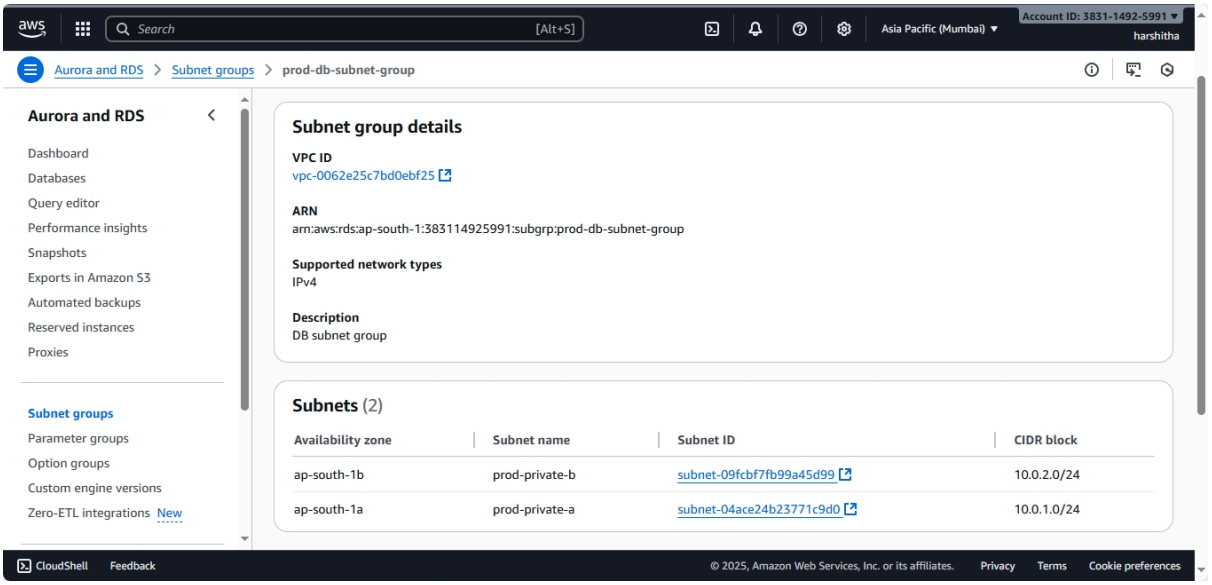
Optional: add CloudWatch Logs endpoint rules or NAT-related egress restrictions.

4.7 Create DB Subnet Group (required for RDS)

Console

1. RDS → Subnet groups → Create DB subnet group.
    - Name: `prod-db-subnet-group`.
    - VPC: `prod-vpc`. Add `prod-private-a` and `prod-private-b`. Save.





CLI

```
aws rds create-db-subnet-group --db-subnet-group-name prod-db-subnet-group \
  --db-subnet-group-description "RDS subnet group for prod" \
  --subnet-ids <SUBNET_ID_A> <SUBNET_ID_B>
```

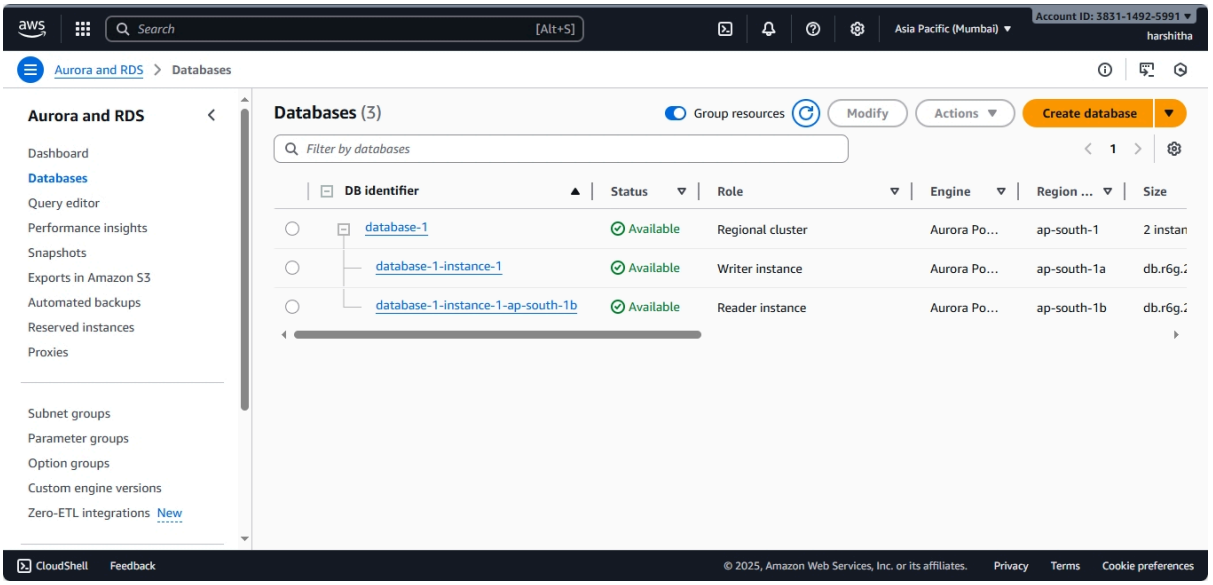
4.8 Create RDS (MySQL) — production settings

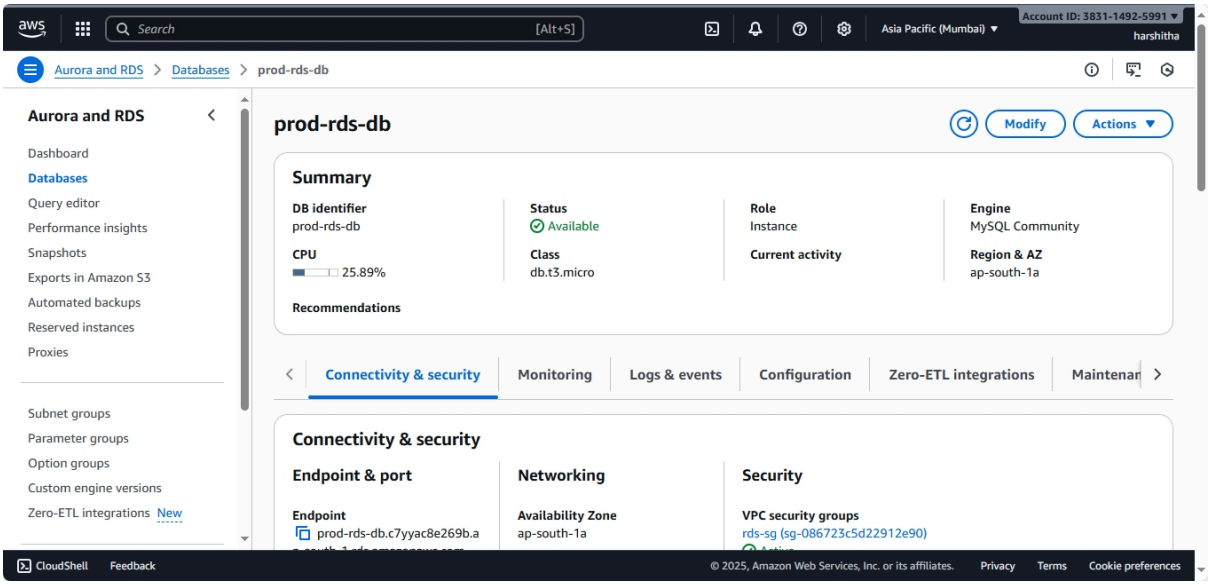
Console (recommended for first time)

1. RDS → Databases → Create database → Standard Create.
2. Engine: **MySQL**. Choose version (e.g., 8.x).
3. Templates: Production (or Free tier if testing).
4. DB instance identifier: `prod-rds-db`.
5. Credentials: `admin` + strong password (or use Secrets Manager option — recommended).
6. DB instance size: For production choose appropriate (e.g., `db.m5.large`), for practice use `db.t3.micro`.
7. Storage: General Purpose SSD (gp3 recommended for prod), size 20 GiB (adjust as needed). Enable autoscaling storage for prod.
8. Availability & durability: **Multi-AZ deployment** (recommended).

9. Connectivity:
    - VPC: `prod-vpc`
    - Subnet group: `prod-db-subnet-group`
    - Public access: **No** (important)
    - VPC security groups: select `rds-sg`
10. Additional configuration:
    - Initial DB name: `prodapp` (optional)
    - Backups: Enable automated backups, retention 7 days (or as required)
    - Enable encryption (use AWS KMS key; default is fine for practice)
    - Enable Performance Insights (recommended)
    - Enhanced monitoring: enable and choose granularity (60s)
    - Maintenance window: keep default or schedule
11. Create database and wait until `Available`.

Note: For Multi-AZ add `--multi-az` and select larger instance class for production.



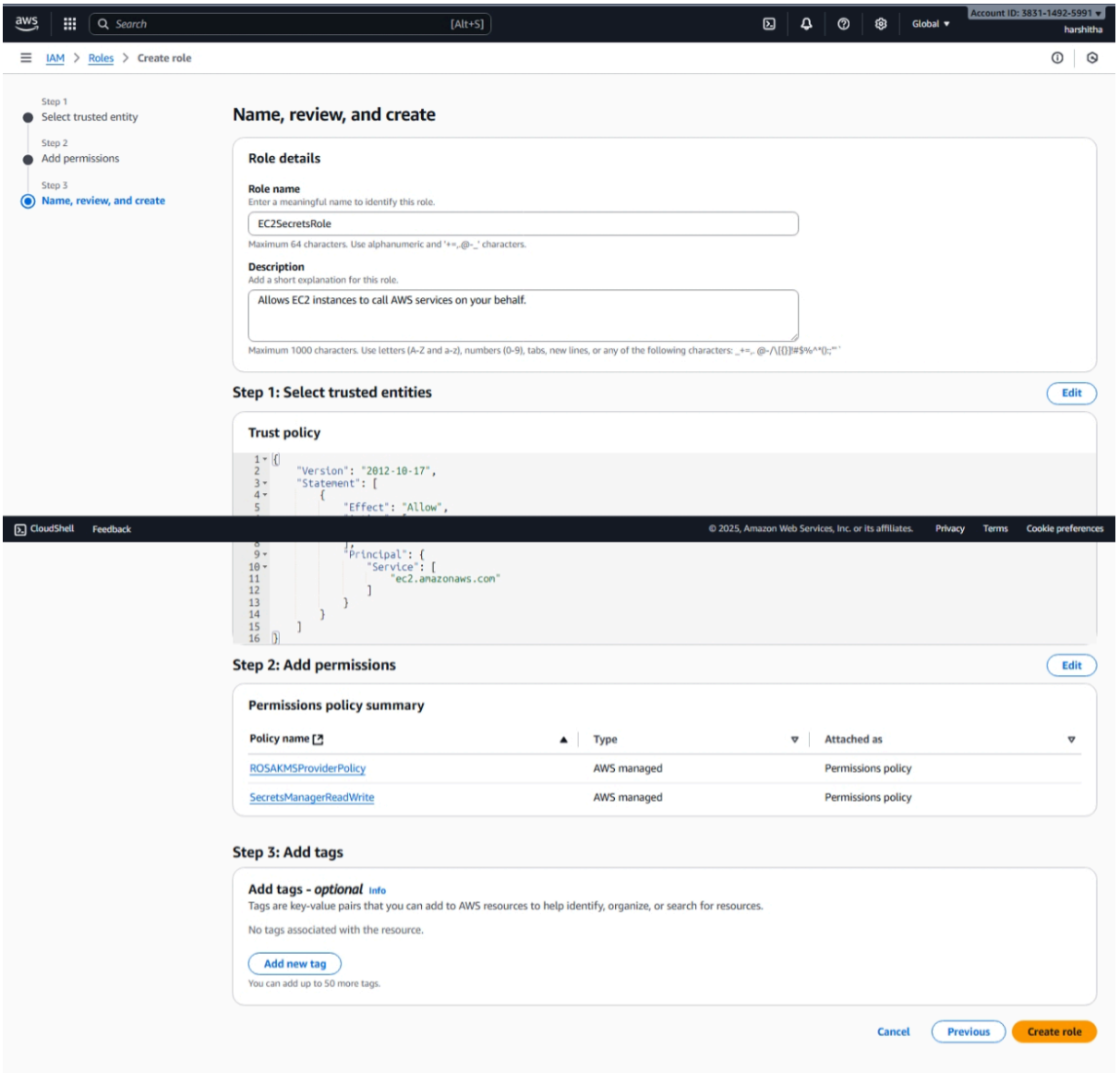


4.9 Create IAM role for EC2 to access Secrets Manager (EC2SecretsRole)

Store DB credentials in Secrets Manager and let EC2 fetch them — avoids hardcoded DB passwords.

Console: create IAM Role

1. IAM → Roles → Create role → AWS service → EC2 → Next.
2. Attach inline/custom policy (least privilege): allow `secretsmanager:GetSecretValue` for secret `prod/rds/master` and `kms:Decrypt` on KMS key if encryption is used.
3. Name role `EC2SecretsRole`. Create.

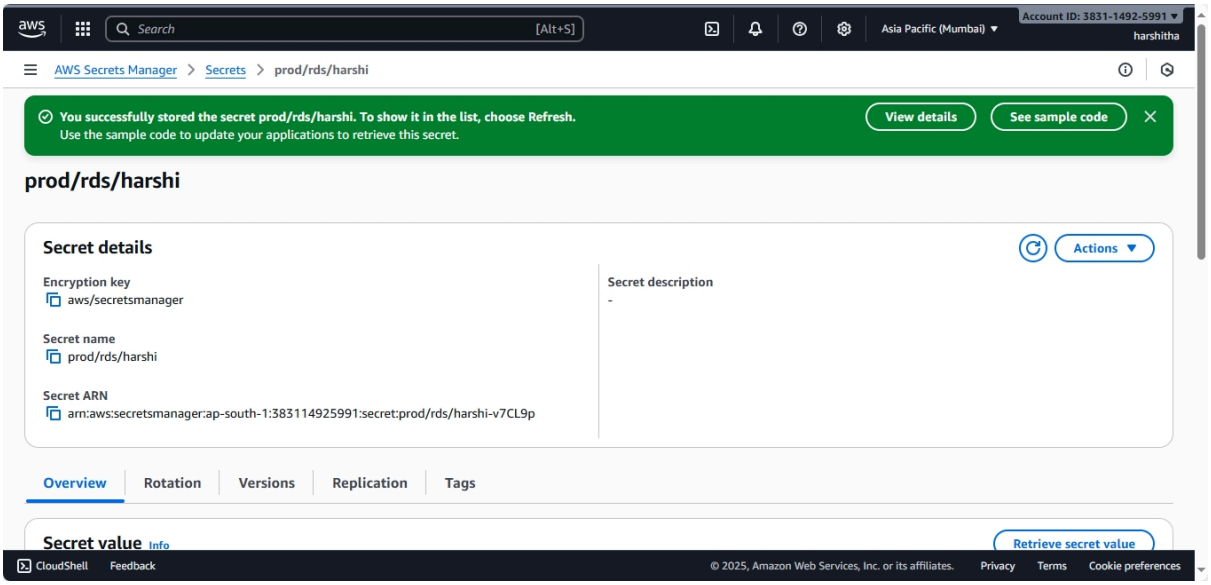


4.10 Store DB credentials in Secrets Manager

Console

1. Secrets Manager → Store a new secret → choose “Credentials for RDS database” or “Other type of secret”.

2. Name: `prod/rds/harshi`. Save.



CLI

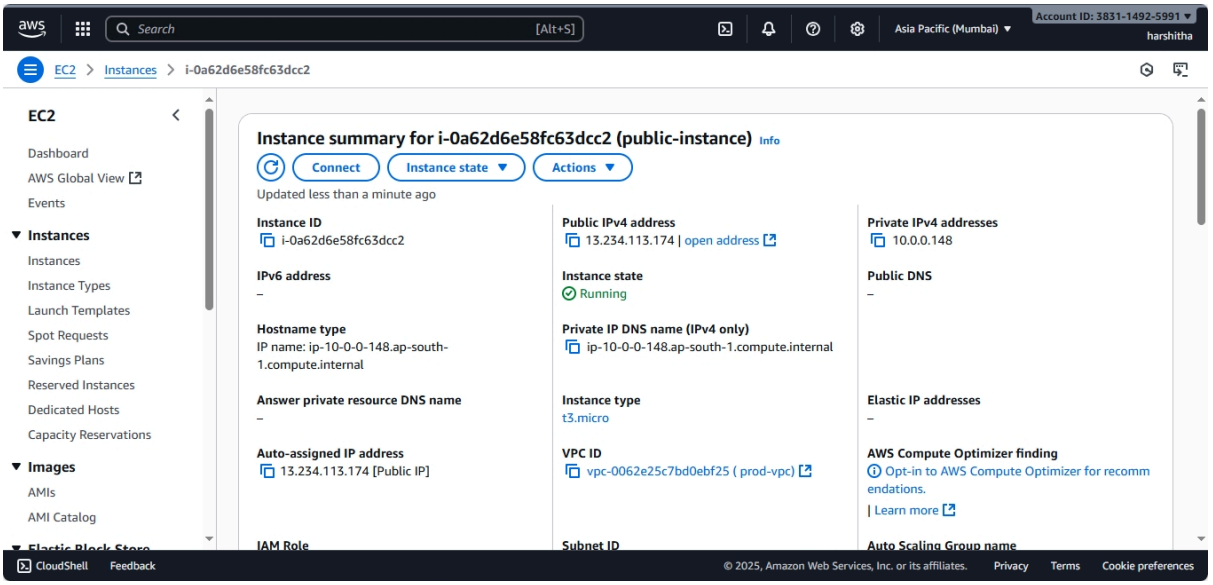
```
aws secretsmanager create-secret --name prod/rds/master \
  --description "RDS master credentials" \
  --secret-string '{"username":"admin","password":"StrongPassw0rd!"}'
```

Ensure IAM role `EC2SecretsRole` has permission to get this secret.

4.11 Launch EC2 bastion and attach role

Console

1. EC2 → Launch instances → choose Amazon Linux → `t3.micro` for testing.
2. Network: VPC `prod-vpc`, Subnet `prod-public-a`, Auto-assign public IPv4: enable.
3. Key pair: select or create
4. Security group: `sg-bastion`.
5. IAM role: select `EC2SecretsRole`.
6. Launch.



CLI

```
aws ec2 run-instances --image-id <AMI_ID> --count 1 --instance-type t3.micro \
  --key-name <KEY_NAME> --subnet-id <PUBLIC_SUBNET_ID> --iam-instance-profile Name=EC2SecretsRole \
  --security-group-ids <SG_BASTION_ID> --associate-public-ip-address
```

Step 1: SSH into Bastion or Connect (UI)

ssh -i mykey.pem ec2-user@<EC2_PUBLIC_IP>

- Replace `mykey.pem` with your key pair file.
- Replace `<EC2_PUBLIC_IP>` with your Bastion EC2's public IP.
- Now you're inside Bastion → from here we'll connect to RDS.

Step 2: Install MySQL Client + AWS CLI + jq

On Amazon Linux 2023, the `mysql` package name is different. Use:

```
sudo yum update -y

sudo yum install -y mariadb105 jq
```

(If `mariadb105` not found, try `mariadb106`)

Verify:

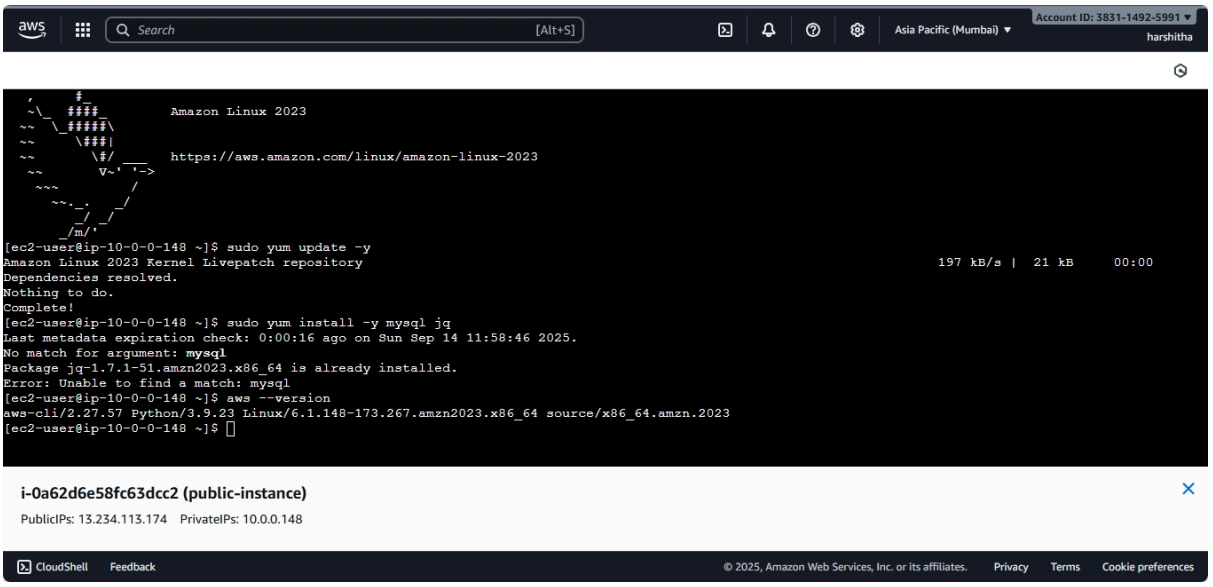
`mysql --version`

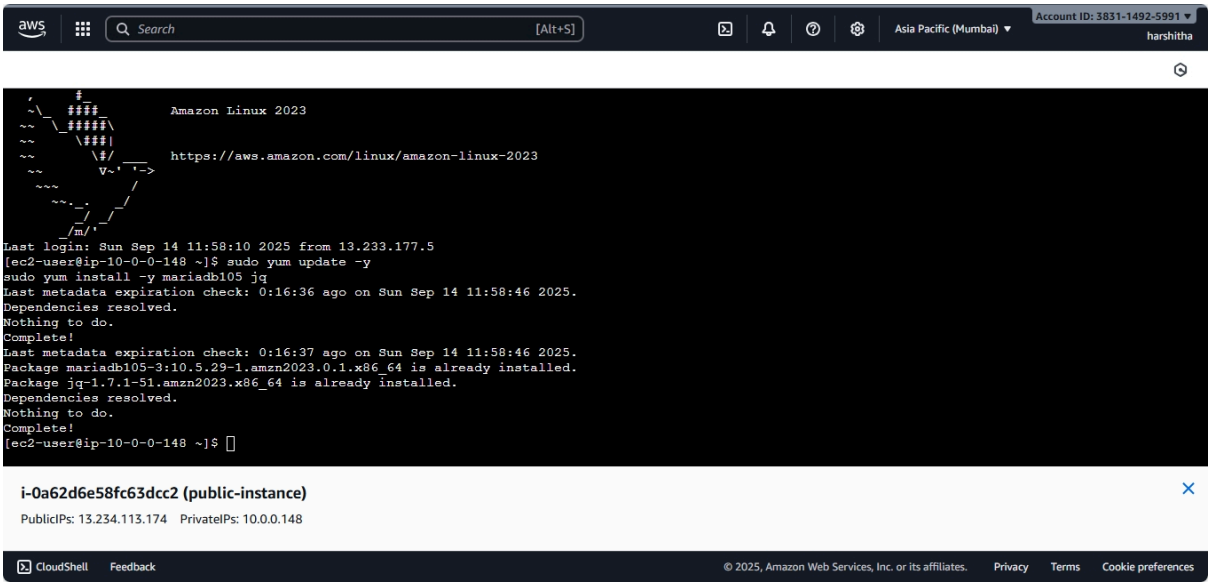
- `mariadb` → required to connect to RDS.
- `jq` → JSON parser, needed to extract username/password from Secrets Manager.

Check AWS CLI:

```
aws –version
```

If not available, install per AWS CLI official docs.





Step 3: Retrieve DB credentials from Secrets Manager

Here’s the secure way (instead of hardcoding passwords):

SECRET_JSON=$(aws secretsmanager get-secret-value \

  --secret-id prod/rds/harshi\

  --region ap-south-1 \

  --query SecretString \

  --output text)

USERNAME=$(echo $SECRET_JSON | jq -r .username)

PASSWORD=$(echo $SECRET_JSON | jq -r .password)

Replace `prod/rds/harshi` with your secret name.
If you’re using the AWS-managed one, replace with something like:

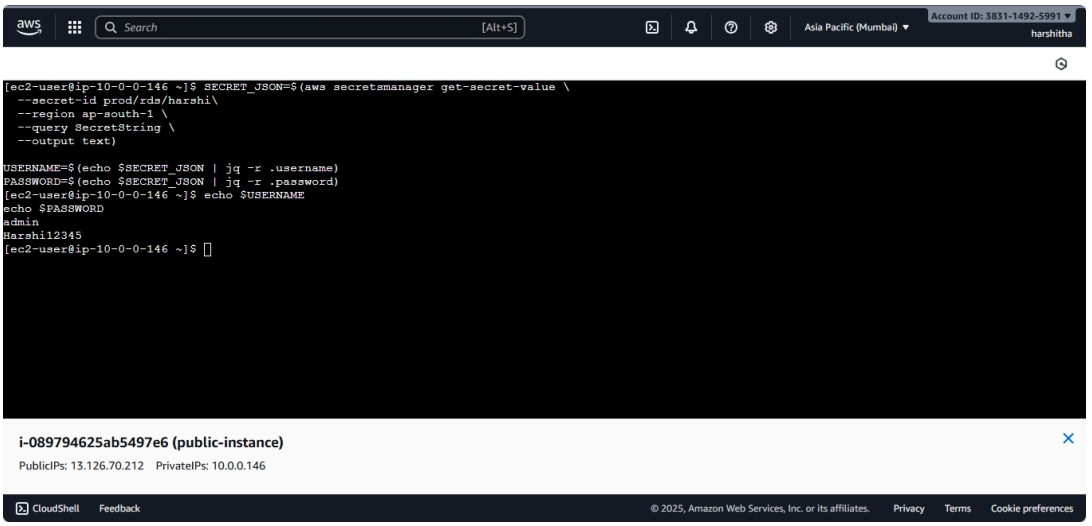
rds!cluster-bf1f6d23-c79c-4215-aecc-70e57c63f9e7

- Replace `ap-south-1` with your region.

Test if variables are set:

echo $USERNAME

echo $PASSWORD



Step 4: Connect to RDS

Get RDS Endpoint from **RDS Console → Connectivity & security → Endpoint**.

RDS_ENDPOINT=prod-rds-db.c7yyac8e269b.ap-south-1.rds.amazonaws.com(replace your end point)

mysql -h $RDS_ENDPOINT -u $USERNAME -p"$PASSWORD"

If successful, you'll land in `mysql>` prompt. Example:

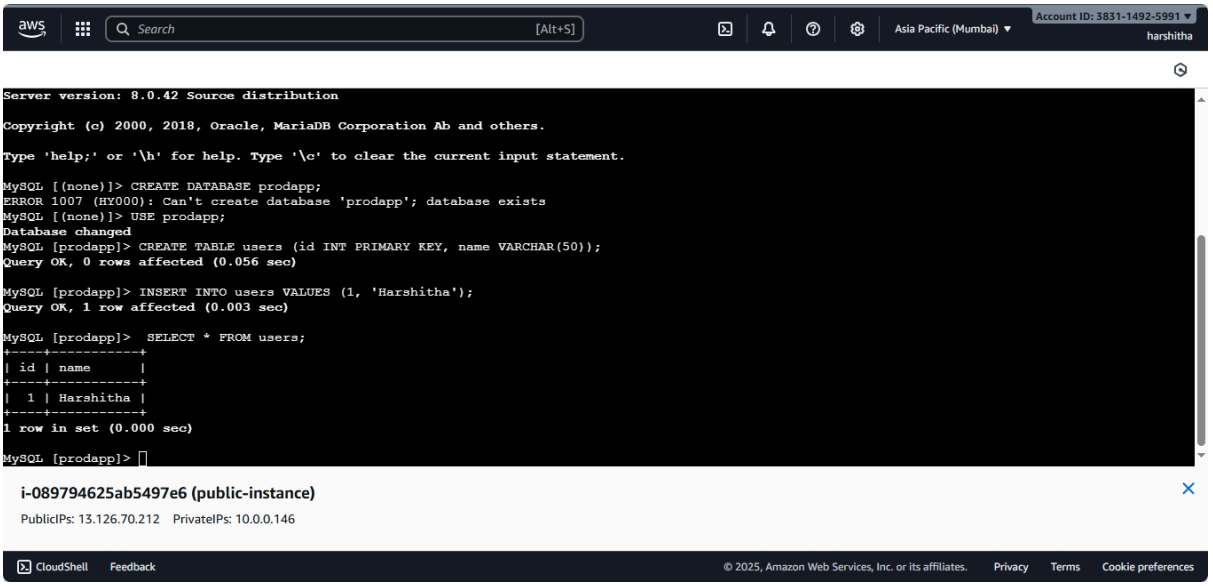
mysql> CREATE DATABASE prodapp;

mysql> USE prodapp;

mysql> CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(50));

mysql> INSERT INTO users VALUES (1, 'Harshitha');

mysql> SELECT * FROM users;



4.12 Optional — Export snapshot to S3 (for archiving/analytics)

If you want to export snapshot data to S3 (Parquet), create an export role and start export.

Create S3 bucket `prod-rds-exports-<suffix>` with encryption (SSE-KMS recommended).

Create IAM role `rds-snapshot-export-role` with trust for `rds.amazonaws.com` and policy allowing PutObject on the bucket and KMS encrypt actions. Example policy provided earlier in the conversation; attach to the role.

Start export (CLI)

```
aws rds start-export-task \
  --export-task-identifier prod-export-1 \
  --source-arn arn:aws:rds:<REGION>:<ACCOUNT_ID>:snapshot:prod-rds-snap-1 \
  --s3-bucket-name prod-rds-exports-<suffix> \
  --iam-role-arn arn:aws:iam::<ACCOUNT_ID>:role/rds-snapshot-export-role \
  --kms-key-id arn:aws:kms:<REGION>:<ACCOUNT_ID>:key/<KMS_KEY_ID>
```

Monitor with `aws rds describe-export-tasks`.

5) Tests & validation (exact commands + SQL)

5.1 Connectivity tests

- From bastion (works):
    - `mysql -h <RDS_ENDPOINT> -u admin -p` → enter password → should connect.
- From your laptop (outside VPC) — expected fail (if public access disabled). If you deliberately try, expect a connection timeout or refused.

5.2 CRUD tests (SQL)

```
CREATE DATABASE testdb;
USE testdb;
CREATE TABLE items (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100));
INSERT INTO items (name) VALUES ('alpha'), ('beta');
SELECT * FROM items;
```

5.3 Snapshot & restore test

- Console → RDS → Databases → Select `prod-rds-db` → Actions → Take snapshot. Name `prod-rds-snap-1`.
- Restore snapshot → Actions → Restore snapshot → New DB id `prod-rds-restore-1`.
- Connect to `prod-rds-restore-1` and verify data.

5.4 Export snapshot to S3 test (if configured)

- Start export, wait completion, check S3 bucket for objects.

6) Troubleshooting — common errors & fixes

- **Cannot connect (timeout)**
    - Check RDS `Publicly accessible` = No (if private).
    - Confirm `sg-rds` inbound port 3306 allows source `sg-bastion`.
    - Confirm bastion is in public subnet with a public IP.
    - Verify route tables: public subnet associated with `public-rt`, private with `private-rt`.
    - If using NAT, ensure NAT is `Available`.
- **Access denied (authentication)**
    - Wrong username/password — verify Secrets Manager secret and that EC2 role can fetch.
    - If using IAM auth — ensure DB user exists and policy allows `rds-db:connect`.
- **Snapshot export fails**
    - Verify `rds-snapshot-export-role` has S3 PutObject, ListBucket and KMS permissions.
    - Ensure S3 bucket encryption/KMS configuration matches the role permissions.

7) Cleanup (essential to avoid charges)

Console steps for safe teardown (recommended order):

1. Stop/terminate `prod-bastion` EC2 instance.

2. Delete RDS instances (choose whether to keep final snapshot). For test: delete with `Skip final snapshot = Yes`.

3. Delete DB snapshots (if not needed).

4. Delete S3 export bucket contents then bucket.

5. Delete route tables, detach and delete Internet Gateway.

6. Delete subnets.

7. Delete VPC.

8. Remove IAM roles and policies (EC2SecretsRole, snapshot export role).

9. Delete Secrets Manager secrets (rotate if needed before deletion).

CLI cleanup examples

aws rds delete-db-instance --db-instance-identifier prod-rds-db --skip-final-snapshot
aws rds delete-db-snapshot --db-snapshot-identifier prod-rds-snap-1
aws ec2 delete-nat-gateway --nat-gateway-id <NAT_ID>
aws ec2 release-address --allocation-id <EIP_ALLOC_ID>
aws s3 rm s3://prod-rds-exports-<suffix> --recursive
aws s3 rb s3://prod-rds-exports-<suffix>
aws ec2 delete-subnet --subnet-id <SUBNET_ID>
aws ec2 detach-internet-gateway --internet-gateway-id <IGW_ID> --vpc-id <VPC_ID>
aws ec2 delete-internet-gateway --internet-gateway-id <IGW_ID>
aws ec2 delete-vpc --vpc-id <VPC_ID>
aws iam delete-role --role-name EC2SecretsRole